Multiproceso con Python

Hilos en Python con threading

Python proporciona el módulo threading que permite trabajar con hilos de manera sencilla. Este módulo es una capa de abstracción sobre la biblioteca de bajo nivel _thread.

```
import threading
```

El módulo threading ofrece:

- La clase Thread para crear hilos
- Diversas herramientas de sincronización (locks, semáforos, eventos, etc.)
- Métodos para controlar la ejecución de hilos

Es importante mencionar que, debido al Global Interpreter Lock (GIL) de Python, los hilos no pueden ejecutarse realmente en paralelo en operaciones intensivas de CPU. Sin embargo, son muy útiles para operaciones limitadas por entrada/salida (I/O) como acceso a archivos, bases de datos o comunicaciones de red.

Ejemplo básico de uso de hilos en Python

```
import threading
import time

def tarea():
    print("Inicio de la tarea")
    time.sleep(2)
    print("Fin de la tarea")

hilo = threading.Thread(target=tarea)
hilo.start()
hilo.join()
print("Hilo finalizado")
```

El código es un ejemplo básico del uso de hilos en Python:

Primero se importan los módulos necesarios:

- threading: proporciona las clases para trabajar con hilos.
- time: se utiliza para crear pausas en la ejecución.

Se define una función tarea() que:

- Imprime un mensaje de inicio
- Espera 2 segundos (simulando una tarea que requiere tiempo)
- Imprime un mensaje de finalización.

Se crea un objeto hilo utilizando threading.Thread(), indicando mediante el parámetro target qué función debe ejecutarse en el hilo.

Se inicia el hilo con hilo.start(), lo que ejecuta la función tarea() en un hilo separado.

Con hilo.join() el programa principal espera a que el hilo termine su ejecución antes de continuar.

Finalmente, se imprime "Hilo finalizado" una vez que el hilo ha terminado

Hilos con argumentos

Este código muestra cómo pasar parámetros a una función que se ejecuta en un hilo en Python:

Hilo con argumentos en Python

```
def saludo(nombre):
    print(f"Hola {nombre}")

hilo = threading.Thread(target=saludo, args=("Luis",))
hilo.start()
hilo.join()
```

Se define una función saludo() que:

- Recibe un parámetro nombre
- Imprime un mensaje personalizado con el nombre recibido

Se crea un objeto hilo usando threading.Thread():

- El parámetro target especifica la función saludo que se ejecutará.
- El parámetro args contiene una tupla con los argumentos que se pasarán a la función. Es importante notar la coma después de "Luis" para crear una tupla de un solo elemento.

Se inicia el hilo con hilo.start(), lo que ejecutará la función saludo() con el argumento "Luis".
Con hilo.join() el programa principal espera a que el hilo termine antes de continuar.
La principal diferencia con el ejemplo anterior es la forma de pasar argumentos a la función. En Python, para pasar parámetros a una función en un hilo, debemos usar:

- args=(param1, param2, ...) para argumentos posicionales
- kwargs={"param1": valor1, "param2": valor2, ...} para argumentos con nombre

El uso de la coma en args=("Luis",) es necesario para que Python interprete correctamente la sintaxis como una tupla de un solo elemento.

Ejecutar múltiples hilos

Ejecutar múltiples hilos en Python

```
def trabajo(n):
    print(f"Trabajando en hilo {n}")
    time.sleep(1)

hilos = []
for i in range(5):
    t = threading.Thread(target=trabajo, args=(i,))
    t.start()
    hilos.append(t)

for h in hilos:
    h.join()
```

Este código muestra cómo crear y gestionar múltiples hilos que se ejecutan en paralelo. Vamos a examinar detalladamente cada parte:

Se define una función trabajo() que:

- Recibe un parámetro n que identifica al hilo
- Imprime un mensaje indicando en qué hilo está trabajando
- Pausa la ejecución durante 1 segundo con time.sleep(1)

Se crea una lista vacía hilos para almacenar las referencias a los hilos

- El primer bucle for:
  - Itera 5 veces (índices del 0 al 4)
  - Para cada iteración:
    - Crea un nuevo hilo que ejecutará la función trabajo con el índice actual como argumento
    - Inicia el hilo inmediatamente con t.start()
    - Guarda la referencia al hilo en la lista hilos
- El segundo bucle for:
  - Recorre todos los hilos almacenados en la lista
  - Para cada hilo llama a h.join(), esperando a que termine su ejecución

Al ejecutar este código verás mensajes como:

```
Trabajando en hilo 0
Trabajando en hilo 1
Trabajando en hilo 2
Trabajando en hilo 3
Trabajando en hilo 4
```

Estos mensajes probablemente aparecerán casi simultáneamente, ya que todos los hilos se inician uno tras otro sin esperar a que terminen. Después de aproximadamente 1 segundo (el tiempo de espera definido), todos los hilos finalizarán.

Aspectos importantes a destacar:

- Concurrencia real: A diferencia de los ejemplos anteriores donde creábamos y esperábamos un hilo antes de continuar, aquí los 5 hilos se ejecutan concurrentemente.
- Control de finalización: El segundo bucle con los join() garantiza que el programa principal no terminará hasta que todos los hilos hayan completado su ejecución.
- Almacenamiento de referencias: Es importante guardar las referencias a los hilos en la lista para poder hacer join() después.
- Paralelismo limitado: Aunque Python permite crear múltiples hilos, debido al GIL (Global Interpreter Lock), los hilos de Python no pueden ejecutar código Python en paralelo en múltiples núcleos. Sin embargo, para operaciones de E/S como este ejemplo con time.sleep(), los hilos son muy efectivos para mejorar el rendimiento.

Este patrón es muy común cuando necesitas realizar varias tareas similares en paralelo, como descargar múltiples archivos, procesar varios elementos de datos, o realizar consultas a bases de datos.

Concurrencia con múltiples hilos: Sincronización con Lock

Un lock es el mecanismo más simple de sincronización. Proporciona exclusión mutua, permitiendo que solo un hilo pueda acceder a un recurso compartido a la vez.

Sincronización de hilos con `Lock` en Python

```
lock = threading.Lock()
contador = 0

def incrementar():
    global contador
    for _ in range(100000):
        with lock:
            contador += 1

h1 = threading.Thread(target=incrementar)
h2 = threading.Thread(target=incrementar)

h1.start()
h2.start()
h1.join()
h2.join()

print("Contador:", contador)
```

Este código muestra un concepto fundamental en programación concurrente: la sincronización de hilos mediante un mecanismo de bloqueo (Lock). Vamos a analizar detalladamente este ejemplo:

Se crea un objeto lock usando threading.Lock():

- Este objeto permite controlar el acceso a recursos compartidos entre hilos
- Solo un hilo a la vez puede adquirir el bloqueo

Se define una variable global contador inicializada a 0:

- Esta es el recurso compartido que será modificado por múltiples hilos

Se define la función incrementar():

- Utiliza la variable global contador
- Realiza 100,000 incrementos
- Para cada incremento, utiliza with lock: para adquirir el bloqueo antes de modificar el contador

Se crean dos hilos h1 y h2:

- Ambos ejecutan la misma función incrementar()
- Si se ejecutan concurrentemente sin sincronización, podrían interferir entre sí
- Se inician ambos hilos y se espera a que terminen con join()
- Finalmente, se imprime el valor final del contador

Al ejecutar este código verás:

```
Contador: 200000
```

Esto demuestra que ambos hilos han incrementado el contador 100,000 veces cada uno, sin pérdida de operaciones.

La importancia del Lock

El uso del objeto lock es crucial en este ejemplo. Aquí está por qué:

Problema de condición de carrera: Sin el bloqueo, podría ocurrir lo siguiente:

- Hilo 1 lee el valor de contador (ejemplo: 50)
- Hilo 2 lee el mismo valor (50)
- Hilo 1 incrementa su copia a 51 y la guarda
- Hilo 2 incrementa su copia a 51 y la guarda
- ¡El resultado sería 51 en lugar de 52!

Operación atómica: El bloque with lock: garantiza que todo el código dentro del bloque se ejecute como una operación atómica:

- Un hilo adquiere el bloqueo
- Realiza la operación completa (lectura + incremento + escritura)
- Libera el bloqueo
- Solo entonces otro hilo puede adquirir el bloqueo y hacer lo mismo

Contexto with: El uso de with lock: es una forma elegante de manejar bloqueos en Python:

- Adquiere automáticamente el bloqueo al entrar al bloque
- Garantiza que el bloqueo se libere al salir del bloque, incluso si ocurre una excepción
- Sin el uso del bloqueo, el valor final del contador sería impredecible y casi siempre menor que 200,000 debido a las condiciones de carrera entre los hilos.

Este patrón es fundamental cuando múltiples hilos necesitan modificar datos compartidos sin interferencias mutuas, garantizando la integridad de los datos en entornos concurrentes.

Uso de queue.Queue para comunicación segura

Patrón Productor-Consumidor en Python

```
import queue

cola = queue.Queue()

def productor():
    for i in range(5):
        print(f"Produciendo {i}")
        cola.put(i)
        time.sleep(0.5)

def consumidor():
    while True:
        item = cola.get()
        if item is None: break
        print(f"Consumiendo {item}")
        cola.task_done()

t1 = threading.Thread(target=productor)
t2 = threading.Thread(target=consumidor)

t1.start()
t2.start()
t1.join()
cola.put(None)
t2.join()
```

Este código implementa el clásico patrón de diseño Productor-Consumidor utilizando hilos y una cola sincronizada. Este es uno de los patrones más importantes en programación concurrente.

Se importa el módulo queue y se crea una cola con queue.Queue():

- Esta es una cola thread-safe, diseñada específicamente para comunicación segura entre hilos
- Maneja automáticamente la sincronización necesaria

Se define la función productor():

- Genera 5 elementos (los números del 0 al 4)

Para cada elemento:

- Imprime un mensaje indicando qué está produciendo
- Coloca el elemento en la cola con cola.put(i)
- Espera 0.5 segundos antes de producir el siguiente elemento

Se define la función consumidor():

- Se ejecuta en un bucle infinito
- Obtiene elementos de la cola con cola.get()
- Si recibe un valor None, sale del bucle (señal de terminación)
- Procesa el elemento (imprime un mensaje)
- Marca la tarea como completada con cola.task_done()

Se crean e inician los hilos productor y consumidor
Se espera a que el productor termine con t1.join()
Se envía None a la cola como señal para que el consumidor termine
Se espera a que el consumidor termine con t2.join()

Al ejecutar este código verás algo así:

```
Produciendo 0
Consumiendo 0
Produciendo 1
Consumiendo 1
Produciendo 2
Consumiendo 2
Produciendo 3
Consumiendo 3
Produciendo 4
Consumiendo 4
```

El orden exacto puede variar ligeramente dependiendo de la programación de los hilos, pero generalmente verás que cada elemento producido es consumido poco después.

Características importantes

- Sincronización automática: La clase Queue maneja internamente los locks necesarios para evitar condiciones de carrera
- Comunicación entre hilos: La cola actúa como un canal de comunicación seguro entre los hilos productor y consumidor
- Patrón de señalización: El valor None se usa como una señal especial para indicar al consumidor que debe terminar
- Método task_done(): Este método es importante cuando se usa join() en la cola, ya que permite a la cola saber cuándo se ha procesado completamente un elemento
- Desacoplamiento: El productor y consumidor no necesitan conocerse entre sí; solo interactúan con la cola

Aplicaciones prácticas

- Este patrón es extremadamente útil en muchos escenarios:
- Procesamiento de tareas en segundo plano
- Sistemas de mensajería
- Procesamiento de eventos
- Gestión de recursos limitados (pool de conexiones, etc.)
- Sistemas de procesamiento por lotes donde la generación y el procesamiento pueden ocurrir a diferentes velocidades

La implementación con Queue en Python hace que sea relativamente sencillo implementar este patrón de manera segura para hilos, evitando problemas comunes como condiciones de carrera o interbloqueos.

Sockets con hilos: servidor y cliente multicliente

Servidor y Cliente con hilos en Python

Servidor con threading

```
import socket
import threading

def atender_cliente(conn, addr):
    print(f"Conectado con {addr}")
    while True:
        data = conn.recv(1024)
        if not data:
            break
        conn.sendall(data.upper())
    conn.close()

def servidor():
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind(("localhost", 8080))
    s.listen()
    print("Servidor escuchando en el puerto 8080")

    while True:
        conn, addr = s.accept()
        hilo = threading.Thread(target=atender_cliente, args=(conn, addr))
        hilo.start()

servidor()
```

Cliente

```
import socket

s = socket.socket()
s.connect(("localhost", 8080))

s.sendall(b"hola desde cliente")
data = s.recv(1024)

print("Recibido:", data.decode())
s.close()
```

Semáforos

Un semáforo es un contador que puede ser incrementado o decrementado por los hilos. Permite limitar el número de hilos que pueden acceder a un recurso simultáneamente.

Control de acceso con `Semaphore` en Python

```
import threading
import time
import random

impresora = threading.Semaphore(3)

def usar_impresora(usuario_id):
    print(f"Usuario {usuario_id} espera para usar la impresora.")
    with impresora:
        print(f"Usuario {usuario_id} está usando la impresora.")
        tiempo_impresion = random.uniform(1, 3)
        time.sleep(tiempo_impresion)
        print(f"Usuario {usuario_id} ha terminado en {tiempo_impresion:.2f} segundos.")

hilos = []
for i in range(10):
    t = threading.Thread(target=usar_impresora, args=(i,))
    t.start()
    hilos.append(t)

for t in hilos:
    t.join()

print("Todos los usuarios han terminado de imprimir.")
```